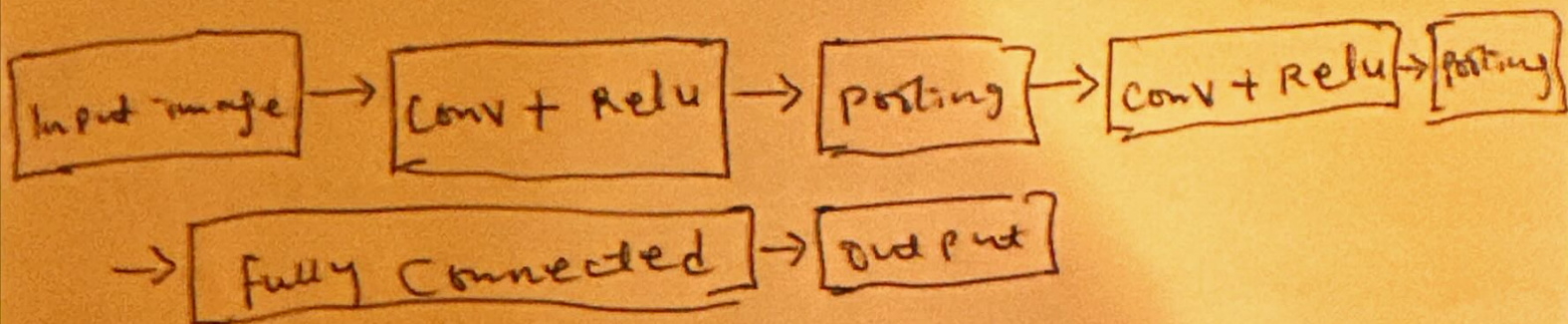


Part 1

Q3

A basic CNN architecture for image classification includes:

- Input Layer: Takes the input image (e.g. grayscale 25×25).
- Convolutional Layers: Apply filters to extract features (e.g. edges).
- Activation functions (e.g. ReLU): Introduce non-linearity.
- Pooling Layers (e.g. Max pooling): Reduce spatial dimensions.
- Fully connected Layers: Combine features for classification.
- Output Layer: produces class probabilities (e.g. dog vs. cat)

Part 2

— Stride: Bob (with 100×100 images) is more likely to use a larger stride. Larger strides reduces the output size of convolutional and pooling layers, which is beneficial for layer images to manage computational complexity and prevent overfitting by reducing feature map sizes early. Alice's smaller 25×25 images requires smaller strides to preserve sufficient spatial information, as aggressive downsampling could lose critical details.

Kernel size: Bob is also more likely to use larger kernel size. Larger kernels capture broader contextual features, suitable for larger images where patterns (e.g., dog vs. cat features) may span more pixels. Alice's smaller images benefit from smaller kernels (e.g., 3×3) to focus on fine-grained details, as larger kernels might over too much of the image, losing specificity. (2)

Q4

The given linear ~~classification~~ SVM decision function is $f(x) = w^T x + b$, with classification rule: predict class +1 if $f(x) \geq 0$, else class -1, designed for binary classification.

Can it classify three classes (car, pedestrian, tree)?

NO, the base SVM model cannot directly classify three classes, as it is designed for binary classification (two classes: +1 or -1). It separates data with a single hyperplane, which cannot partition the feature space into three distinct regions.

Strategy for three-class classification:

Use a One-vs-Rest (OVR) approach: Train three binary SVM classifiers: Classifier 1: Car (+1) vs (Pedestrian, Tree) (-1). Classifier 2: Pedestrian (+1) vs (Car, Tree) (-1). Classifier 3: Tree (+1) vs (Car, Pedestrian) (-1).

For a new input x , compute decision function $f_1(x), f_2(x), f_3(x)$ for each classifier. Assign the class with

the highest decision score: $\arg \max_i f_i(x)$. (3)

This leverages the binary SVM model by training multiple classifiers, each distinguishing one class from the others, enabling multi-class classification.

Q6

Watershed Segmentation combines ideas from region growing and edge detection. Segments an image into several "catchment basins" or "regions". The watershed algorithm treats an image as a topographic surface, where pixel intensities represent heights. The key idea is to simulate flooding ~~when~~ from local minima (low intensity regions), with "water" rising until boundaries (watershed lines) form between catchment basins, segmenting the ~~regions~~ image into regions.

Non-Marker Based Watershed

Floods from every local minimum in image. Requires only the gradient magnitude image (or the image itself). No guidance or constraints - every small intensity dip becomes a seed. Very prone to over-segmentation, especially with noisy data.

Marker Based Watershed

Starts flooding only from selected points (markers). Markers can be user-defined or generated via preprocessing (e.g., thresholding, distance transform).

Helps control region growth and avoid over-segmentation. (4)

This is the improved and practical version used in most real-world applications.

It also requires user input or preprocessing to place markers but produces more controlled results.

Q5 ~~Q8~~

Part 1

The full form of HOG is Histogram of oriented Gradients.

HOG features Extraction process:

1. Preprocess image: Convert image to grayscale and optionally normalize (e.g gamma correction)
2. Compute Gradients: Calculate the gradient magnitude and orientation at each pixel using filters (e.g sobel). For pixel $I(x, y)$, gradients are $G_x = I(x+1, y) - I(x-1, y)$, $G_y = I(x, y+1) - I(x, y-1)$.

Magnitude: $\sqrt{G_x^2 + G_y^2}$, Orientation: $\arctan(G_y/G_x)$

3. Divide into cells: Partition the image into small cells (e.g 8×8 pixels).
4. Build Histograms: For each cell, create a histogram of gradient orientations, weighted by magnitude, over discrete bins (e.g 9 bins for $0-180^\circ$).
5. Block Normalization: Group cells into larger blocks (e.g 2×2 cells). Normalize gradient magnitudes

with each block (e.g. L_2 -norm) to reduce illumination effects.

6. Concatenate Features: Collect normalized histogram from all blocks to form the final HOG feature vector.

Part 2: Performance of textured vs non-textured objects

HOG performs better on textured objects. HOG relies on gradient orientations, which are rich in textured regions with varying intensities (e.g. edges, patterns). Textured objects, like a leopard's fur, yield more informative features due to distinct gradient patterns. Non-textured objects, like a smooth sphere, have fewer gradients, producing sparse histograms that are less discriminative. For example, in object recognition, a textured image of a dog's fur provides more robust HOG features than a non-textured image of a plain wall.

(48)

(6)

Traditional Segmentation models determine annotations by analyzing inherent qualities of each pixel (called "heuristics") like color and intensity.

Fast and efficient for basic tasks like semantic segmentation.

It also rely on hand-crafted features and classical image processing techniques (e.g thresholding, edge detection or watershed algorithms)

Deep learning segmentation models employ complex neural networks for sophisticated pattern recognition.

More precise and robust for complex tasks like panoptic segmentation.

End-to-end training optimizes features extraction and segmentation simultaneously

Part 1

Autoencoders: Autoencoders are neural networks designed for unsupervised learning, consisting of an encoder and a decoder. The encoder compresses the input (e.g. image) into a lower-dimensional, latent representation, capturing essential features. The decoder reconstructs the input from this representation. For classification the latent ~~features~~ representation can be used as features for a classifier. The key idea is learning data representations by minimizing ~~representations~~ reconstruction loss.

Vision Transformers (ViT): ViTs adapt the Transformer architecture for images by dividing the images into fixed-size ~~pattern~~ patches (e.g. 16x16 pixels), flattening each ~~pixel~~ ^{patch} into a vector, and processing them as a sequence with self-attention mechanisms.

Part 2

For a resource-constrained device classifying medical image as ~~benign~~ ^{benign} or malignant an autoencoder is likely the better choice. Autoencoders are ~~computationally~~ computationally lighter, as they involve simpler feed-forward networks for encoding and decoding and the latent ~~classification~~ representation can be used with a lightweight classifier.

Q1

8

The min filter takes the minimum value in each 2×2 block of the image. The image is 4×4 , so the output will be $\frac{4}{2} \times \frac{4}{2} = 2 \times 2$

Block 1 (top-left, indices $(0,0)$ to $(1,1)$): Values are $[2, 4, 1, 12]$. Minimum is $\min(2, 4, 1, 12) = 1$

Block 2 (top-right, indices $(0,2)$ to $(1,3)$): Values are $[3, 7, 13, 5]$. Minimum is $\min(3, 7, 13, 5) = 3$

Block 3 (bottom-left, indices $(2,0)$ to $(3,1)$): Values are $[0, 3, 7, 3]$. Minimum is $\min(0, 3, 7, 3) = 0$

Block 4 (bottom-right, indices $(2,2)$ to $(3,3)$): Values are $[5, 2, 11, 7]$. Minimum is $\min(5, 2, 11, 7) = 2$

$$\text{Output} = \begin{bmatrix} 1 & 3 \\ 0 & 2 \end{bmatrix}$$

Q2

9

To maintain the same output size (4×4), we assume zero-padding around the input image (one pixel border, since the kernel is 3×3). Correlation involves sliding the kernel over the image, multiplying corresponding elements, and summing the results. The kernel is not rotated (unlike convolution).

The padding image (with zero) is:

$$\begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 0 \\ 0 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Let's compute the correlation for each position of the kernel's center over the original image indices (x, y) from $(0, 0)$ to $(3, 3)$. The kernel's non-zero elements are at positions $(0, 0)$, $(0, 2)$, $(1, 0)$, $(1, 2)$, $(2, 0)$, $(2, 2)$, all with value 1. Hence, the correlation at position (x, y) is the sum of pixel intensities at positions (x, y) , $(x, y+2)$, $(x+1, y)$, $(x+1, y+2)$, $(x+2, y)$, $(x+2, y+2)$ in the padded image.

Manually computing for each position (noting that image indices are x (column), y (row) starting at 0):

(10)

For $(x=0, y=0)$: positions map to image pixels
at $(0,0), (0,2), (1,0), (1,2), (2,0), (2,2)$: Values $[1, 1, 0, 1, 1, 1]$

Sum $= 1 + 1 + 0 + 1 + 1 + 1 = 5$.

Similarly for all positions.

Hence the output will be:

Output =

5	4	5	4
3	4	5	5
4	3	5	5
2	2	4	4